



Chapter 9

Real Time Interrupts and Hardware Interfacing

Introduction to Hardware Interrupts

- **Definition:**
 - Real interrupts generated by external hardware.
- **INT Pin:**
 - Single external pin on the processor to receive hardware interrupts.
- **Purpose:**
 - Ensure real-time response to external events (e.g., key presses, printer status).

Why Hardware Interrupts?

- **Interrupts vs. Polling:**
 - **Polling:** Actively monitor devices, inefficient for the processor.
 - **Interrupts:** Notify the processor only when needed, enabling multitasking.
- **Examples:**
 - Keyboard: Stores keypress during long computations.
 - Printer: Signals when ready for new data.

Programmable Interrupt Controller (PIC)

- **Purpose:**
 - Manages multiple interrupt signals and forwards them to the processor.
- **Features:**
 - 8 input lines (IRQ 0–7), 1 output signal to processor.
 - Assigns priorities to IRQ lines (0 = highest priority).
- **Cascading PICs:**
 - IBM AT systems: Two PICs provide 16 interrupt sources (IRQ 0–15).

Priority Management in PIC

- **How it Works:**
 - Forward highest-priority interrupt to the processor.
 - Lower-priority interrupts are queued until the higher-priority interrupt is serviced.
- **End Of Interrupt (EOI):**
 - Sent by the processor to signal completion of interrupt handling.



Interrupt Requests (IRQ) Lines and Their Functions

IRQ	Function	Interrupt
IRQ 0	Timer device	INT 8
IRQ 1	Keyboard	INT 9
IRQ 2	Cascade signal for 2nd PIC	INT A
IRQ 3	Serial port (COM 2)	INT B
IRQ 4	Serial port (COM 1)	INT C
IRQ 5	Sound card/network card	INT D
IRQ 6	Floppy disk drive	INT E
IRQ 7	Parallel port	INT F

IRQ to INT Mapping

- **Mapping by PIC:**

- IRQ lines (0–7) → INT 8–F.
- Example:
 - IRQ 0 → INT 8 (Timer interrupt).
 - IRQ 1 → INT 9 (Keyboard interrupt).

- **Processor Perspective:**

- Interrupt number is automatically generated by PIC without extra instructions.

Example: Timer Interrupt (IRQ 0 → INT 8)

- **Timer Chip:**
 - Generates interrupts at precise intervals.
 - Handler updates system time.
- **Example Code Snippet:**

```
; Timer Interrupt Handler Example
MOV AL, 20h          ; EOI signal for PIC
OUT 20h, AL          ; Send EOI to PIC
IRET                 ; Return from interrupt
```

Handling Keyboard Interrupt (IRQ 1 → INT 9)

- **Keypress Handling:**
 - Stores keypress for later use.
- **Replacing the Interrupt Vector:**
 - Custom handler can be used to modify behavior.

- **Example Code Snippet:**

; Keyboard Interrupt Handler Example

IN AL, 60h	; Read keypress from keyboard port
MOV [keybuffer], AL	; Store keypress
MOV AL, 20h	; EOI signal
OUT 20h, AL	; Send EOI to PIC
IRET	; Return from interrupt

keybuffer DB ?	; Buffer for storing keypress
----------------	-------------------------------

Key Takeaways

- **Hardware Interrupt Flow:**
 - IRQ → PIC → INT Pin → Processor → Interrupt Handler.
- **Important Concepts:**
 - IRQ to INT mapping.
 - EOI signal for enabling lower-priority interrupts.
- **Best Practices:**
 - Properly handle and reset IRQs to avoid conflicts.

Introduction to I/O Ports

- **What are I/O Ports?**
 - Mechanism for processor to communicate with peripheral devices.
 - Supports **read** and **write** operations.
- **Memory vs. I/O Ports**
 - **Memory:** Stores program data.
 - **Ports:** Perform hardware control or interactions.

Peripheral Address Space

- **IBM PC Design:** Separate memory and peripheral address spaces.
- **Memory Mapped I/O:** Some processors map ports to memory cells.
- **Address Bus Usage:** Lower 16 bits are used for I/O ports (65,536 possible ports).

IN and OUT Instructions

- **Purpose:** Read/write data from/to ports.
- **Key Points:**
 - MOV: Access memory space.
 - IN and OUT: Access peripheral address space.

Example Instructions:

```
in al, 0x21           ; Read from port 21
out 0x21, al           ; Write to port 21
mov dx, 0x378
in al, dx              ; Read from port in DX
out dx, al             ; Write to port in DX
```

PIC Ports

- **Ports 20 and 21:**
 - Port 20: **Control Port.**
 - Port 21: **Interrupt Mask Register (IMR).**
- **Interrupt Masking:**
 - Bit 0-7: Correspond to IRQ lines.
 - **0:** Enable interrupt.
 - **1:** Disable interrupt.

Example Program: Disable Keyboard Interrupt

- **Purpose:** Disable IRQ 1 for the keyboard.
- **Code:**

```
in al, 0x21          ; Read IMR
or al, 2             ; Set bit for IRQ 1
out 0x21, al         ; Write back to IMR
mov ax, 0x4c00       ; Terminate program
int 0x21
```
- **Result:** Keyboard interrupts are stopped.

Keyboard Scan Codes

- **Key Features:**
 - Each key press/release sends a **scan code**.
 - Press Code: Most Significant Bit (MSB) = 0.
 - Release Code: MSB = 1.
- **Examples:**
 - **Shift-A Sequence:**
 - Shift Press Code
 - A Press Code
 - A Release Code
 - Shift Release Code

Interrupt Chaining

- **Definition:** Allows original ISR to function after custom ISR completes.
- **Steps:**
 - Save the address of the original ISR.
 - Jump to it at the end of the custom ISR using JMP FAR.

Terminate and Stay Resident (TSR) Programs

- **Purpose:** Retain part of a program in memory after termination.
- **Difference in Termination Methods:**
 - INT 21h, Service 4C: Releases all memory.
 - INT 21h, Service 31: Retains a portion of memory.

Memory Allocation for TSR

- **DOS Memory Structure:**
 - Reserved Areas: Interrupt Vector Table, BIOS/DOS data, COMMAND.COM.
 - **Transient Program Area:** For loaded programs.
- **Freemem Pointer:** Adjusted to allocate or reclaim memory.

Calculating Resident Memory

- **Steps:**
 - Place a label at the end of the required code.
 - Calculate bytes to retain.
 - Round up using: $(\text{bytes} + 15) / 16$.
- **Example:**
 - 100 bytes $\rightarrow$ 7 paragraphs (rounded).

TSR Program Example

- **Goal:** Display L for left shift and R for right shift.
- **Features:**
 - Captures scan codes.
 - Prevents shift keys from being forwarded to BIOS.
- **Command:** Verify resident program using `mem /c`.

Conclusion

- **I/O Ports:** Enable hardware communication and control.
- **IN/OUT Instructions:** Facilitate port data manipulation.
- **TSR Programs:** Extend operating system functionality by retaining memory.

Introduction to Programmable Interval Timer (PIT)

- **What is PIT?**
 - A peripheral device used to generate precise time intervals.
 - Chip model: **8254**.
- **Fixed Frequency:**
 - Input frequency: **1.19318 MHz**.
 - Independent of processor clock speed.

Divisor and Timer Tick

- **16-bit Divisor:** Divides the input frequency to control the output.
- **Special Case:** Divisor of 0 → **65536**.
- **Default Behavior:**
 - Divisor: **65536**.
 - Timer Tick Frequency: **18.2 Hz** ($1193180/65536$).
 - Interval Between Ticks: **55ms**.

PIT and System Time

- **Purpose:**
 - Maintains system time with precise intervals.
 - Triggers **IRQ 0** to handle system clock.
- **High Priority Interrupt:**
 - Breaks execution of other processes.
 - Used for time-sensitive operations.

PIT in TSR Programs

- **Integration with TSR:**
 - TSR programs rely on interrupt hooking for activation.
 - PIT's precise interrupts are ideal for periodic tasks.
- **Timer Interrupt:**
 - Hooked to perform custom actions (e.g., updating a tick count).

Example: Hooking Timer Interrupt

Objective: Hook the timer interrupt to display tick count.

Code:

```
[org 0x0100]          ; Origin for emu8086
jmp start              ; Jump to start of the program

old_isr dw 0           ; Placeholder for original interrupt vector

start:
    ; Save old timer interrupt vector
    push es
    mov ax, 0
    mov es, ax
    mov bx, 0x1C * 4    ; Interrupt 1Ch vector
    mov ax, es:[bx]
    mov [old_isr], ax
    mov ax, es:[bx + 2]
    mov [old_isr + 2], ax

    ; Hook new interrupt
    cli
    lea ax, new_isr
    mov es:[bx], ax
    mov ax, cs
    mov es:[bx + 2], ax
    sti
```

```
; Terminate and Stay Resident
```

```
mov dx, end - start + 15
```

```
shr dx, 4                ; Convert bytes to paragraphs
```

```
mov ah, 0x31
```

```
int 0x21
```

```
new_isr:
```

```
; Custom handler
```

```
push ax
```

```
push cx
```

```
push dx
```

```
; Increment tick count
```

```
mov dx, tick_count
```

```
inc word [dx]
```

```
; Restore old handler
```

```
call far [old_isr]
```

```
pop dx
```

```
pop cx
```

```
pop ax
```

```
iret
```

```
tick_count dw 0          ; Counter to hold tick count
```

```
end:
```

Features:

- Hooks **INT 1Ch** (timer interrupt).
- Updates and displays a tick counter.

Key Points to Remember

- **PIT Features:**
 - Fixed input frequency ensures precision.
 - Generates periodic IRQ 0 interrupts.
- **TSR Integration:**
 - Control is interrupt-driven, never directly returned.
 - PIT enables periodic tasks for TSRs.

Conclusion

- **PIT:** A versatile peripheral device for system timing.
- **Assembly Programming:** Provides direct access to PIT functionality.
- **TSR Usage:** Combines with PIT for interrupt-driven applications.

Introduction to Parallel Ports

- **Purpose:**
 - Enable computers to control external hardware.
 - Examples: Controlling LEDs, sending signals to printers, or other devices.
- **Types of Ports:**
 - Parallel Port, Serial Port, USB, FireWire.
- **Focus:** Parallel Port (DB-25).

Parallel Port Connector

- **Connector:**
 - **DB-25:** A 25-pin connector.
 - Bidirectional communication.
- **Pin Overview:**
 - Pins **2-9**: Data pins (Processor → Device).
 - Pin **10 (ACK)**: Acknowledges receipt of data.
 - Pins **18-25**: Ground (Electrical reference point).

Parallel Port Control

- **Processor View:**
 - Processor communicates via parallel port controller, not directly with pins.
- **Ports for LPT1:**
 - **378:** Data port (sends data to pins 2-9).
 - **37A:** Control port (manages IRQ and ACK signals).
- **Voltage Levels:**
 - **1 bit:** 5V on corresponding pin.
 - **0 bit:** 0V on corresponding pin.

Control Port (37A)

- **Bit 4:** Enables **IRQ 7** triggering via **ACK (Pin 10)**.
- **Common Applications:**
 - Sending scancodes of pressed keys to external hardware (e.g., LEDs).

Circuit Example: LED Control

- **Circuit Setup:**
 - 8 LEDs connected to data pins (Pins 2-9).
 - Ground pins (18-25) connected to circuit ground.
- **Functionality:**
 - LEDs light up based on the data sent to port **378**.
 - Binary data from the processor is converted to voltage signals.

Example Program: Sending Scancodes to LEDs

```
[org 0x0100]          ; Origin for emu8086
jmp start              ; Jump to start of program

start:
    ; Keyboard hook setup
    mov ah, 0x09        ; INT 09H keyboard interrupt
    mov dx, old_int
    int 0x21             ; Save old interrupt vector
    mov ah, 0x25        ; Set new interrupt vector
    lea dx, new_isr
    int 0x21
```

```
    ; Stay resident
    mov dx, end - start + 15
    shr dx, 4            ; Convert bytes to paragraphs
    mov ah, 0x31
    int 0x21
new_isr:
    ; Read scan code
    in al, 0x60          ; Read keyboard scan code
    out 0x378, al        ; Send to parallel port
    iret                 ; Return from interrupt
old_int dw 0             ; Placeholder for original ISR address
end:
```

Key Points to Remember

- **Parallel Port Pins:**
 - Pins 2-9: Data transmission.
 - Pin 10 (ACK): Interrupt signaling (IRQ 7).
- **Voltage Representation:**
 - 1 = 5V, 0 = 0V.
- **Practical Example:** Sending keyboard scancodes to LEDs.

Conclusion

- **Parallel Ports:** Essential for interfacing computers with external hardware.
- **Assembly Programming:** Provides direct control of data and signals.
- **Applications:** Light control, printers, custom hardware projects.